ASYNCHRONOUS FIFO

Design & Verification Using SystemVerilog

Industry-Style RTL Implementation for Clock Domain Crossing (CDC)

Key Focus Areas

- Parameterized RTL Architecture
- Gray Code Pointer Synchronization
- Two Flip-Flop Synchronizers
- Full & Empty Flag Generation
- Self-Checking Verification Environment
- Randomized Testbench & Assertions

Tools

- **SystemVerilog**
- **Cadence Xcelium**
- **SimVision**
- **Quartus Prime**

Designed & Verified by

Sarathi Selvam D.

The Problem

Why Do We Need an Asynchronous FIFO?

Digital systems often contain multiple modules operating at **different clock frequencies**. Directly transferring data between these independent clock domains can lead to **metastability**, resulting in unreliable or corrupted data.

An **Asynchronous FIFO** provides a safe and reliable mechanism to transfer data while preserving data integrity.

The Challenge

Without an Asynchronous FIFO

- Data Loss
- Metastability
- Invalid Sampling
- Timing Violations
- Unreliable Communication

With an Asynchronous FIFO

- Safe Clock Domain Crossing (CDC)
- Reliable Data Transfer
- Independent Read & Write Clocks
- High Throughput
- Robust Synchronization

Typical Applications

- Multi-core SoCs
- FPGA Designs
- PCIe Interfaces
- Ethernet Controllers
- UART Bridges
- DMA Controllers
- High-Speed Data Acquisition Systems

RTL Architecture

Asynchronous FIFO Architecture

The design is divided into independent **write** and **read** clock domains. Gray code pointers and two-stage synchronizers enable safe pointer exchange across clock domains, while the dual-port memory allows simultaneous read and write operations.

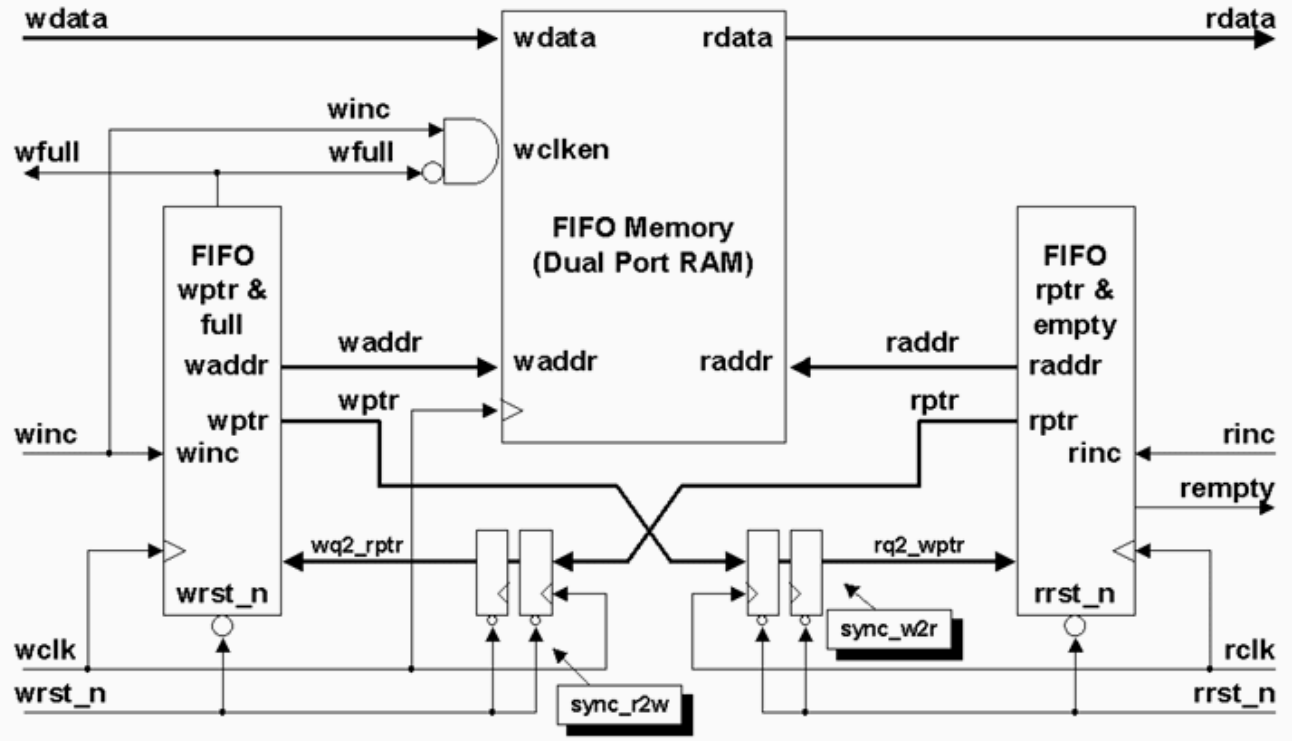


Image: CummingsSNUG2002SJ_FIFO1_rev1_2

Key Design Features

- Modular RTL architecture
- Parameterized FIFO depth and data width
- Independent read and write clock domains
- Gray code pointer synchronization
- Two flip-flop synchronizers for CDC
- Full and empty status flag generation
- Simultaneous read and write support

Module	Function
async_fifo.sv	Top-level integration of all modules
fifomem.sv	Dual-port FIFO memory
wrdomain.sv	Write pointer generation and full flag logic
readomain.sv	Read pointer generation and empty flag logic
sync_r2w.sv	Synchronizes read pointer into write clock domain
sync_w2r.sv	Synchronizes write pointer into read clock domain

Design Choice	Why It Matters
Independent Read & Write Clocks	Supports asynchronous communication
Gray Code Pointers	Only one bit changes per increment, reducing CDC errors
Two-Flip-Flop Synchronizers	Helps resolve metastability before pointer comparison
Dual-Port Memory	Enables simultaneous read and write operations
Parameterized Design	Easily scalable for different FIFO sizes
Modular RTL	Debugging, testing, and reuse

Write Clock Domain

Write Pointer & Full Flag Generation

The write clock domain is responsible for accepting incoming data, updating the write pointer, storing data into the FIFO memory, and detecting when the FIFO becomes full.

Key Components

1. Binary Write Pointer

- Maintains the current write location.
- Increments only when:
  - winc = 1
  - wfull = 0

---

2. Gray Code Conversion

- Binary pointer is converted into Gray code.
- Only one bit changes between consecutive values.
- This minimizes metastability during clock domain crossing.

---

3. Full Flag Logic

The synchronized read pointer is compared with the next Gray-coded write pointer.

When both pointers indicate that the FIFO has reached its maximum capacity, the **Full** flag is asserted, preventing additional writes until data is read.

Design Highlights

- Synchronous write operation
- Binary pointer for addressing memory
- Gray pointer for CDC synchronization
- Full flag prevents overflow
- Parameterized pointer width

Read Clock Domain

Read Pointer & Empty Flag Generation

The read clock domain retrieves data from the FIFO, updates the read pointer, and determines when the FIFO becomes empty. Like the write domain, it uses Gray code pointers for safe synchronization across clock domains.

Key Components

1. Binary Read Pointer

- Tracks the current read location.
- Increments only when:
  - rinc = 1
  - rempty = 0

2. Gray Code Conversion

- Converts the binary pointer into Gray code.
- Ensures only one bit changes at a time.
- Safely transfers pointer information to the write clock domain.

3. Empty Flag Logic

The synchronized write pointer is compared with the next Gray-coded read pointer.

When both pointers become equal, all written data has been read, and the FIFO is declared **Empty**.

Design Highlights

- Independent read clock domain
- Binary pointer for memory addressing
- Gray pointer for CDC synchronization
- Empty flag prevents underflow
- Parameterized pointer width

Clock Domain Crossing (CDC)

Safe Data Transfer Across Independent Clock Domains

In an asynchronous FIFO, the **write clock (wclk)** and **read clock (rclk)** operate independently. Since there is no fixed timing relationship between them, directly transferring binary pointers can lead to **metastability** and incorrect FIFO status detection.

To ensure reliable communication, the design uses **Gray code encoding** and **two-stage flip-flop synchronizers**.

Why Gray Code?

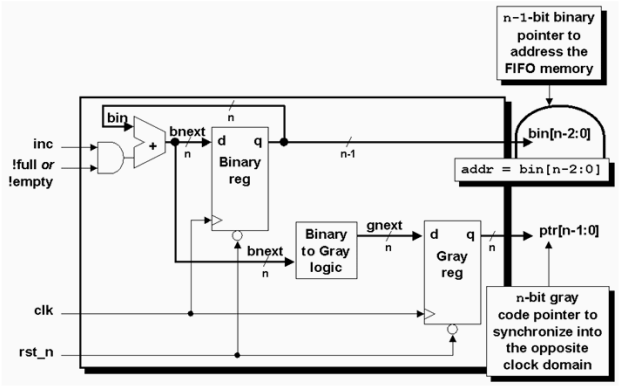
Binary Counter

7 → 8

0111
1000
^^^^

4 bits change simultaneously

Dual n bit gray code counter - style #2



Gray Code Counter

7 → 8

0100
1100
^

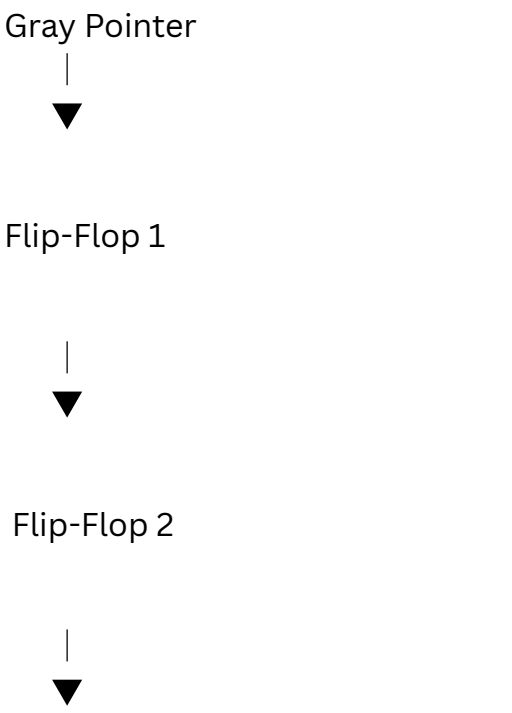
Only ONE bit changes

Benefits of Gray Code

- Only one bit changes per count
- Reduces the chance of incorrect sampling
- Improves reliability during clock domain crossing
- Simplifies full and empty flag generation

Why a Two-Flip-Flop Synchronizer?

The synchronized Gray pointer passes through **two cascaded flip-flops** in the destination clock domain.



Purpose

- Reduces the probability of metastability
- Provides a stable pointer to the destination logic
- Ensures reliable comparison for FIFO status flags

Verification Methodology

Self-Checking Verification Environment

To validate the functionality of the asynchronous FIFO, a **self-checking SystemVerilog testbench** was developed. The verification environment automatically generates transactions, compares expected and actual results, and reports pass/fail without manual waveform inspection.

Verification Components

Clock Generator

- Generates independent write and read clocks
- Mimics asynchronous operation

Reset Generator

- Initializes the FIFO to a known state
- Verifies proper reset behavior

Write Task

- Drives random input data
- Prevents writes when the FIFO is full

Read Task

- Reads data from the FIFO
- Prevents reads when the FIFO is empty

Reference Queue

- Stores the expected sequence of written data
- Acts as the golden reference model

Scoreboard

- Compares expected data with actual FIFO output
- Automatically reports **PASS** or **FAIL**

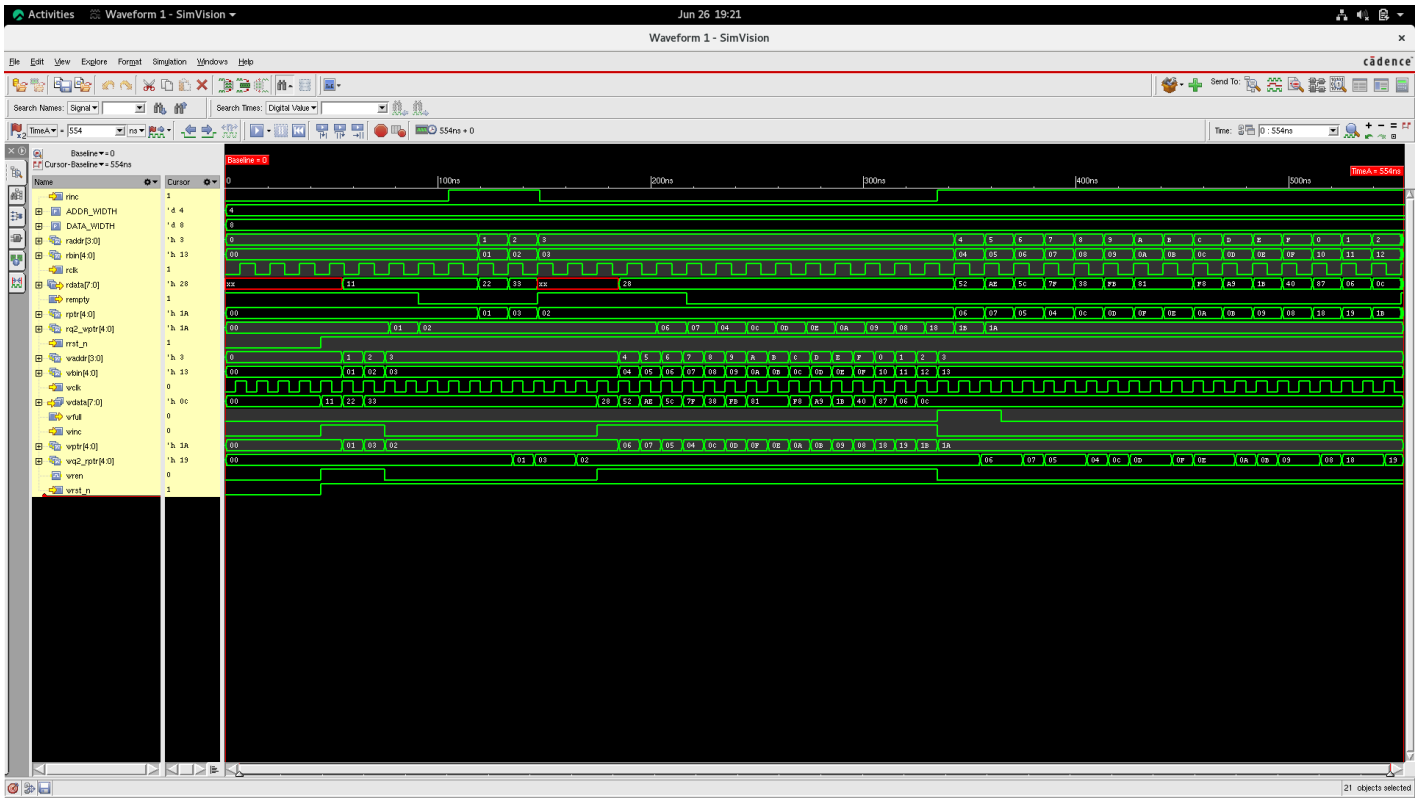
Verification Features

- Self-checking testbench
- Independent clock generation
- Randomized read/write transactions
- Reference queue
- Scoreboard-based checking
- Automatic pass/fail reporting
- Assertion support

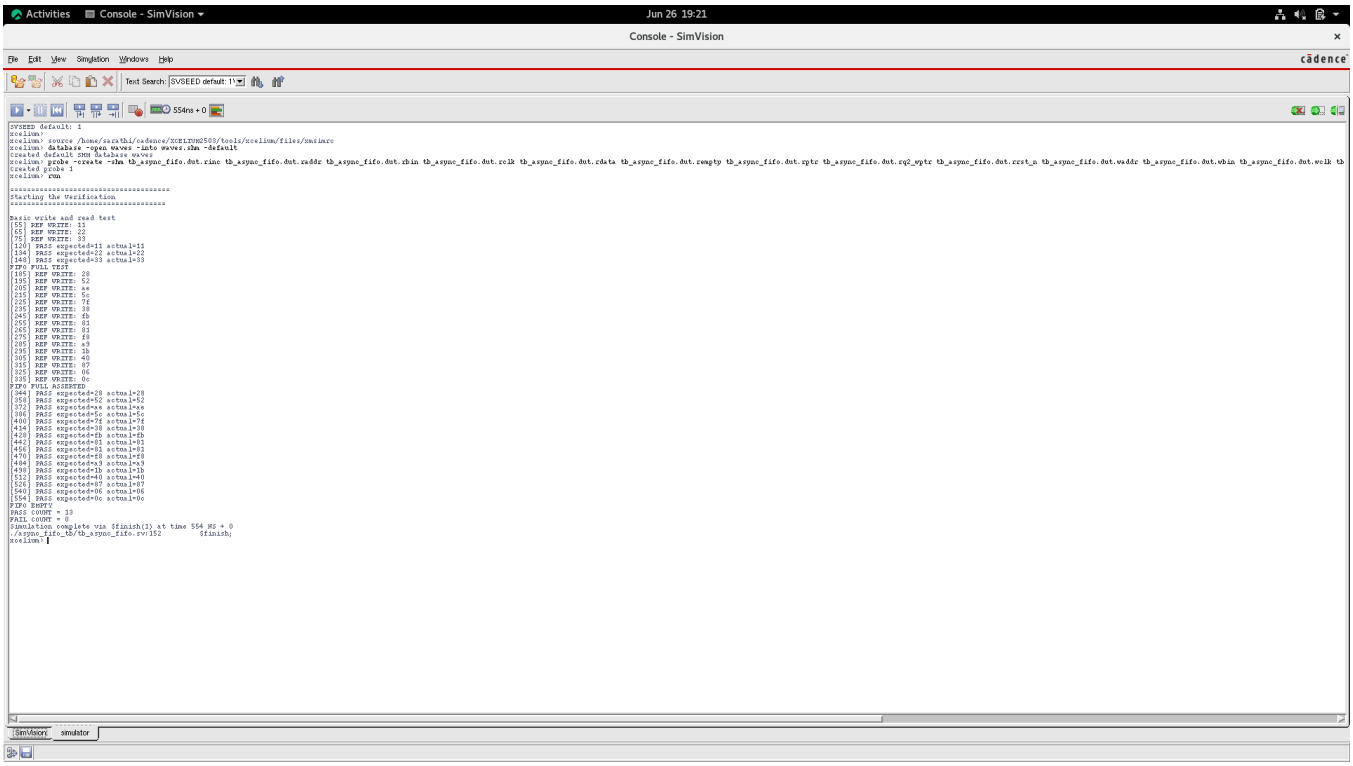
Simulation Results

Functional Verification

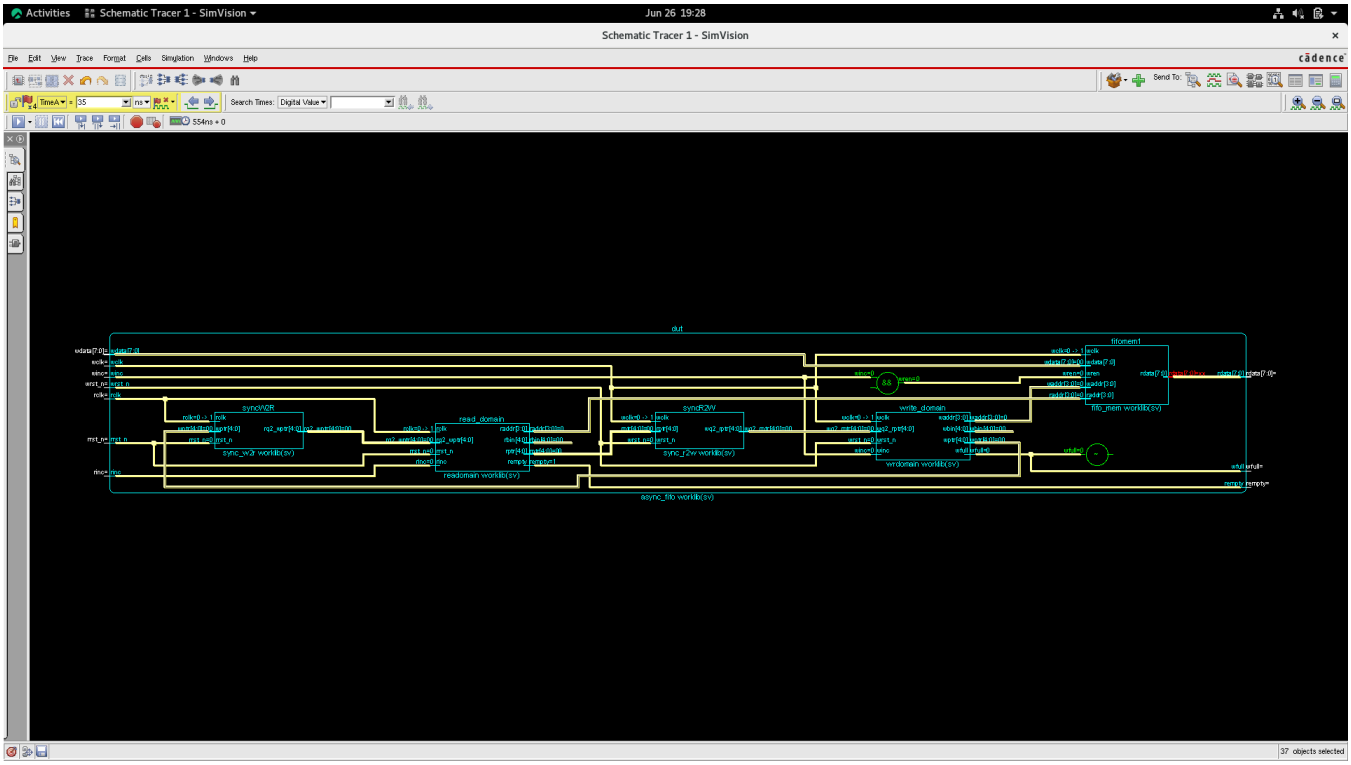
The asynchronous FIFO was functionally verified using **Cadence Xcelium** with a self-checking SystemVerilog testbench. Independent clock domains, randomized transactions, and automated checking were used to validate the design.



Console



Synthetic Tracer



- **Stimulation Tool**: Cadence Xcelium

Test Scenario	Result
Reset Initialization	PASS
Basic Write & Read Operations	PASS
FIFO FULL Detection	PASS
FIFO EMPTY Detection	PASS
Simultaneous Read & Write	PASS
Random Data Transactions	PASS
Data Integrity Verification	PASS

Simulation Summary

- ✔️ Zero data mismatches
- ✔️ Correct FIFO ordering maintained
- ✔️ Reliable operation across asynchronous clock domains
- ✔️ Successful pointer synchronization
- ✔️ Full and Empty flags asserted correctly

Result:

The FIFO transferred data reliably between independent clock domains while maintaining data integrity.

Debugging & Validation Journey

Engineering Challenges Solved

Developing a reliable asynchronous FIFO required careful debugging of both the RTL and the verification environment.

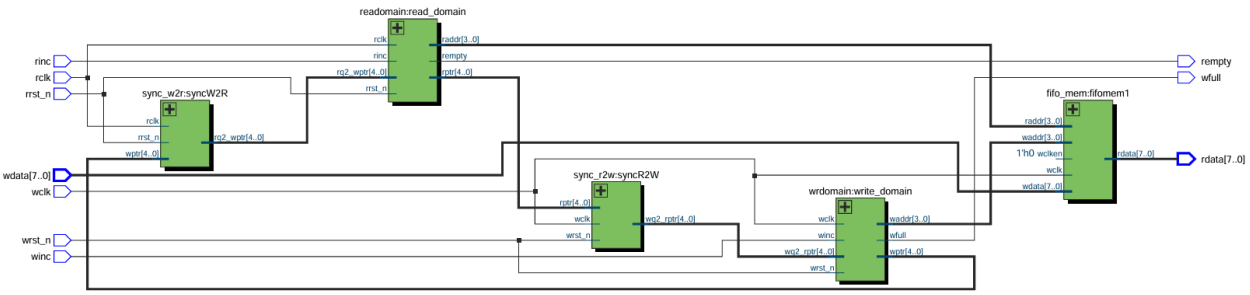
Key Improvements

- Resolved missing RTL signal connections
- Eliminated unknown (X) values during read operations
- Corrected scoreboard comparison timing
- Fixed read transaction timing with respect to rclk
- Removed duplicate comparison logic
- Initialized PASS/FAIL counters
- Accounted for synchronization latency during FULL and EMPTY verification
- Verified pointer wrap-around and FIFO ordering

Engineering Insight

Designing an asynchronous FIFO is not only about implementing RTL—it requires understanding **Clock Domain Crossing (CDC)** behavior, synchronization latency, and robust verification techniques.

Quartus Synthesis



Project Outcome & References

Project Outcome

The asynchronous FIFO was successfully designed, implemented, and functionally verified using **SystemVerilog**. The project demonstrates reliable **Clock Domain Crossing (CDC)** through Gray code pointer synchronization, two-stage synchronizers, and a self-checking verification environment.

Achievements

- Modular and parameterized RTL implementation
- Reliable data transfer across asynchronous clock domains
- Correct FULL and EMPTY flag generation
- Self-checking verification with scoreboard
- Successful randomized functional testing
- Zero functional mismatches during verification

Skills Demonstrated

- RTL Design
- Clock Domain Crossing (CDC)
- Gray Code Synchronization
- SystemVerilog
- Functional Verification
- Debugging & Waveform Analysis
- Cadence Xcelium
- SimVision

References

Books

- Clifford E. Cummings — *Simulation and Synthesis Techniques for Asynchronous FIFO Design* View article
- Samir Palnitkar — *Verilog HDL*

Documentation & Learning Resources

- IEEE Std 1800™ — *SystemVerilog Language Reference Manual (LRM)*
- Cadence Xcelium & SimVision Documentation
- Intel Quartus Prime Documentation

Github:

GitHub - SARATHIselvam/Asynchronous_FIFO_System_Verilog